

Lisandro Córdoba Laguarda
Punto 2 en repo individual
hoja: 1 (ej. 2 virtual)

42

Arquitectura y Organización de Computadoras - DC - UBA

Segundo Parcial
Segundo Cuatrimestre 2024

Promocional

Normas generales

- El parcial es INDIVIDUAL
- Puede disponer de la bibliografía de la materia y acceder al repositorio de código del taller de system programming, desarrollado durante la cursada
- Las resoluciones que incluyan código, pueden usar assembly o C. No es necesario que el código compile correctamente, pero debe tener un nivel de detalle adecuado para lo pedido por el ejercicio.
- Numere las hojas entregadas. Complete en la primera hoja la cantidad de hojas entregadas
- Entregue esta hoja junto al examen. La misma no se incluye en el total de hojas entregadas.

Régimen de Aprobación

- Para aprobar el examen es necesario obtener como mínimo 60 puntos.
- Para promocionar es condición suficiente y necesaria obtener como mínimo 80 puntos tanto en este examen como en el primer parcial

NOTA: Lea el enunciado del parcial hasta el final, antes de comenzar a resolverlo.

Contexto

Nos encargaron el desarrollo de un kernel para la nueva consola Orga Génesis que cuenta con un procesador x86. Para ahorrar trabajo vamos a tomar nuestro kernel y expandirlo para permitir la ejecución de juegos mediante cartuchos, que contendrán el código y los recursos gráficos (sprites de personajes, fondos). Se incorpora entonces un lector de cartuchos que, entre otras cosas, copiará los gráficos del cartucho a un buffer de video de 4 kB en memoria. Cada vez que el buffer esté lleno y listo para su procesamiento, el lector se lo informará al kernel mediante una interrupción externa mapeada al IRQ 40 de nuestro x86.

Primer ejercicio

Distintas tareas se ocuparán de mostrar los gráficos en pantalla y de realizar otros pre/postprocesamientos de imagen on-the-fly.

Para esto, el sistema debe soportar que las tareas puedan acceder al buffer de video a través de los siguientes mecanismos:

- DMA (Direct Memory Access) - se mapea la dirección virtual 0xBABAB000 del espacio de direcciones de la tarea directamente al buffer de video.
- Por copia - se realiza una copia del buffer en una página física específica y se mapea en la dirección virtual 0xBABAB000. Cada tarea debe tener una copia única.

El buffer de video se encuentra en la dirección física de memoria 0xF151C000 y solo debe ser modificable por el lector de cartuchos.

a) Programar una función `void buffer_dma(pd_entry_t* pd)` que dado el page directory de una tarea realice el mapeo del buffer en modo DMA.

b) Programar una función `void buffer_copy(pd_entry_t* pd, paddr_t phys)` que dado el page directory de una tarea realice la copia del buffer a la dirección física pasada por parámetro y realice el mapeo correspondiente.

Segundo ejercicio

Las tareas configuran en memoria una variable que sirve para comunicar al sistema de qué manera van a acceder al buffer de video.

Cada tarea guarda en la dirección virtual 0xACCE50 (mapeada como r/w para la tarea) un `uint8_t` acceso con posibles valores 0, 1 y 2. El valor 0 indica que la tarea no accederá al buffer de video, 1 que accederá mediante DMA y 2 que accederá por copia. De acceder por copia, la dirección virtual donde realizar la copia estará dada por el valor del registro `ecx` al momento de llamar a `opendevise`.

(1) a)

```
void buffer_dma (Pd_entry * Pd) {  
    mmu_map_page (Pd,  
                   (vaddr_t) 0xBABAB000,  
                   (paddr_t) 0xF1S1C000,  
                   (U | P));  
}
```

Esta función mapea la virtual 0xBABAB000 de la tarea "Jueña" de Pd a la dirección física del buffer (0xF1S1C000).

Como tiene que ser accesibles por los tareas pero solo en lectura, van los atributos U y P.

b)

~~void buffer_copy (Pd_entry_t * Pd, paddr_t Phys, vaddr_t Virz)~~

```
void buffer_copy (Pd_entry_t * Pd, paddr_t Phys, vaddr_t Virz) {  
    copy_page (Phys, (paddr_t) 0xF1S1C000);  
    mmu_map_page (Pd, Virz, Phys, (U | P));  
}
```

Realiza la copia del buffer en la Pagina física

pasada por parametro y luego mapea la virtual pasada por parametro a la copia.

Asumo que cuando se accede por copia la tarea tampoco puede escribir en la copia. Solo el kernel actualiza la copia. Por eso, los attrs (U | P)

NOTA

consulte y me dijeron que es así que ~~asumo~~ asumo.

tiene más sentido que la copia se pueda escribir, pero el enunciado no aclaraba ok!

② El Ponto 2 lo subi a git (pero individual)

~~nombre~~
nombre de usuario: ~~1c1azzaro~~ 1c1azzaro ✓

hash commit: 5972f964

EJERCICIO 2

PARTE a

Las tareas guardan en la virt 0xACCE50 (mapeada como w/r) un uint8_t acceso que indica como accedera al buffer

- acceso = 0 -> no accedera al buffer
- acceso = 1 -> accedera mediante DMA
- acceso = 2 -> accedera por copia y la virtual donde realizar la copia estará en ECX cuando se llame a OPENDEVICE()

cuando se completa el buffer, el lector avisa mediante isr40

isr40 debe:

- mapear el buffer a las tareas que acceden por DMA y hayan solicitado acceso al buffer
- actualizar la copia de las tareas que acceden por copia y hayan solicitado acceso al buffer

Hay que definir la entrada en la idt, para eso agregamos en idt.c IDTENTRY0(40) (pues es interrupcion externa)

^{isr}
En irs.asm agregamos

```
global _isr100
_isr40:
    pushad
    call pic_finish1

    call deviceready

    popad
    iret
```

ojo, en ecx tenés la dirección virtual de la copia, la física nunca queda reflejada en ningún lado de la tarea y por eso necesitas crear una estructura donde guardarla. lo resolvés correctamente.

A PARTIR DE ACA -> buffer.c

En phy_copy_buffer_tasks me voy a guardar las direcciones físicas donde hice la copia del buffer para cada tarea ¿POR QUE ES NECESARIO? -> Pq si bien se que cuando la tarea pide opendevise tengo en ^{ecx} eax la dirección virtual donde debo mapearle la copia, en el caso en que tengo que actualizar la copia con la tarea en el estado RUNNABLE ya no tengo el ^{ecx} eax preservado en la pila como para volver a llamar a buffer_copy. La solución es guardarme la dirección física donde tiene guardada la copia para poder actualizarsela con copy_page

En tasks_opendevise me voy a guardar por cada tarea: 1 si la tarea pidió el buffer 0 si no lo pidió (o si lo pidió pero ya llamo a closedevice)

```
paddr_t phy_copy_buffer_tasks[MAX_TASKS] = {0};
uint8_t tasks_opendevise[MAX_TASKS] = {0};
/*
```

También será necesario agregar el estado
TASK_WAITING_BUFFER para las tareas en sched. c

```
*/
```

```
void deviceready(){  
for(int8_t i; task_ID < MAX_TASK; i++){
```

```
uint32_t cr3 = tss_tasks[task_ID].cr3;
```

```
uint8_t acceso = leer_variable_acceso(cr3);
```

→ mal, ver comentarios en código

```
// Solo me importan las tareas que hayan llamado a opendevce y todavia no a closedevice
```

```
if(tasks_opendevce[task_ID] == 1){
```

```
    // Si la tarea tiene acceso == 1
```

```
    // => Mapeo el buffer a la tarea
```

```
    if(acceso == 1){
```

```
        buffer_dma(cr3);
```

```
    }
```

```
    // Si la tarea tiene acceso == 2
```

```
    // => Actualizo la copia del buffer en la tarea
```

```
    else if(acceso == 2){
```

```
        // if(la tarea está en WAITING_BUFFER)
```

```
        if(sched_tasks[task_ID].state == TASK_WAITING_BUFFER){
```

```
            tss_t tss = tss_tasks[task_ID];
```

```
            uint32_t esp = tss.esp;
```

```
            vaddr_t direccion_virtual_de_copia = *(esp + offset_ecx);
```

```
            // (esp+offset_ecx) es la direccion del ecx en la
```

```
            // pila de kernel de la tarea
```

```
            /*
```

OBS: Se que el ecx guardado en la pila de kernel será el correcto ya que la sy.
opendevce va a hacer pushad y luego pondra la tarea en el estado = WAITING_BUFFER
de esta manera no se tocará la pila ya que volvera a estar RUNNABLE recién
cuando la isr40 termine de mapear/copiar el buffer

```
            */
```

```
            paddr_t direccion_fisica_de_copia = obtenerDireccionFisica(cr3, direccion_virtual_de_copia);
```

```
            // ESTA ES LA FUNCION QUE ESTA EN LA DIAPO 17 DE LA CLASE PRE-PARCIAL
```

```
            phy_copy_buffer_tasks[task_ID] = direccion_fisica_de_copia;
```

```
            // Guardo la fisica donde esta la copia para dps poder actualizarla
```

```
            // Si la virtual ya tenia mapeada una copia, usamos la misma fisica para ahorrar memoria
```

```
            if(direccion_fisica_de_copia != 0){
```

```
                buffer_copy(cr3, direccion_fisica_de_copia, direccion_virtual_de_copia);
```

```
            }
```

```
            // Si no tenia mapeada una copia, pido una pagina nueva
```

```
            else {
```

```
                buffer_copy(cr3, mmu_next_free_user_page(), direccion_virtual_de_copia);
```

```
            }
```

```
            // Si la tarea ya esta usando una copia, la actualizo directamente en la pagina fi.
```

```
        } else if (sched_tasks[task_ID].state == TASK_RUNNABLE){
```

```
            copy_page(phy_copy_buffer_tasks[task_ID], 0xF151C000)
```

```
        }
```

```
    }
```

→ sched_task[task_ID].state = TASK_RUNNABLE;

(en comentarios mencionas que hace falta esta linea, pero no lo pusiste)

```
uint8_t leer_variable_acceso(uint32_t cr3){
```

```
uint32_t virt = 0xACE50;
```

```
pd_entry_t* pd = CR3_TO_PAGE_DIR(cr3);
```

```
uint32_t pdi = VIRT_PAGE_DIR(virt);
```

// Obtengo el page directory de la ta.

```
pt_entry_t* pt = (MMU_ENTRY_PADDR(pd[pdi].pt);
```

```
uint32_t pti = VIRT_PAGE_TABLE(virt);
```

// Obtengo la page table correspondie.

```
paddr_t page_del_acceso = MMU_ENTRY_PADDR(pt[pti].page);
```

```
uint8_t* direccion_dato = ((page_del_acceso & 0xFFFFF000) | VIRT_PAGE_OFFSET(virt));
```

```
return direccion_dato[0];
```

```
// Devuelvo el uint8_t guardado en la direccion virtual 0xACE50
```

bien

es una dirección física, al desreferenciarla se va a

PARTE b

interpretar como una virtual y vamos a parar a cualquier lado.

~~Has lo que quieras es~~ Tampoco puedes leer ϕ ACE50 directo. Hay que mapear page-del-acceso a una física temporal como

Hay que definir en idt.c las entradas en la idt para las dos syscalls Van a ser entradas de nivel 3 pues queremos que la en

puedan llamar las tareas de nivel de usuario

copy-page y leer temp ϕ ACE50 (y desmapear temp).

```
✓ IDT_ENTRY3(99); // 99 opendevise IDT_ENTRY3(100); // 100 closedevice
```

En isr.asm hay que definir las rutinas de atencion a las syscalls

```
global _isr99
```

```
_isr99:
```

```
pushad
```

```
// Este es el pushad que me garantiza tener el ecx correcto en la pila de kernel
```

```
// funcion en buffer.c que simplemente hace sched_tasks[current_task] = TASK_WAITING_BUFFER;
```

```
call set_state_waiting
```

```
// funcion en buffer.c que simplemente hace tasks_opendevise[current_task] = 1;
```

```
call open_device
```

```
/*
```

Saltamos a la proxima tarea. Esta no va a volver a ejecutarse hasta que se llene el buffer se ejecute la rutina isr40, se termine de realizar el mapeo/copia del buffer para esta tarea y se la vuelva a marcar como RUNNABLE

```
*/
```

```
call sched_next_task
```

```
mov word [sched_task_selector], ax
```

```
jmp far [sched_task_offset]
```

```
popad
```

```
iret
```

```
global _isr100
```

```
_isr100:
```

```
pushad
```

```
// Este es el pushad que me garantiza tener el ecx correcto en la pila de
```

```
call closedevice
```

```
// funcion en buffer.c que simplemente hace tasks_opendevise[current_task] =  $\phi$ ;
```